

Specification of team software project

Department of Software Engineering

Faculty of Mathematics and Physics, Charles University

Solvers: Jan Krátký

Study program: Computer Science - Software and Data Engineering

Project title: Service Application for Recommender Systems with Sparse Autoencoders

Project type: Research project

Supervisor: Mgr. Ladislav Peška, Ph.D.

Consultants: doc. RNDr. Jakub Lokoč, Ph.D., Mgr. Petr Škoda, Ph.D., RNDr. Patrik Dokoupil,
Mgr. Martin Spišák

1 Introduction

This project is carried out within the research activities of the Department of Software Engineering at the Faculty of Mathematics and Physics, Charles University, and is part of the broader research initiative LLM4MORS, funded by the Czech Science Foundation (GAČR). The research group focuses on leveraging Large Language Models (LLM) in multi-objective recommender systems. The primary aim is to enhance the explanation¹ of recommendations and to facilitate user (or editor) steering². Currently, the LLM4MORS project intensively explores the use of Sparse Autoencoders (SAE) for creating interpretable representations of user profiles within the recommendation process and for enabling associated steering mechanisms.

Despite promising results, the integration of sparse autoencoders into recommender systems presents several challenges. The process typically consists of multiple interconnected steps, including model training, representation learning, neuron interpretation, evaluation, and steering. Each of these steps allows for a variety of alternative methods and experimental configurations. Existing research projects within LLM4MORS and related work (see Section 1.1) address only specific combinations of these steps and are often implemented in an ad-hoc manner. As a result, experimentation is difficult to reproduce, extend, or systematically compare, which limits efficient exploration of the overall problem space.

My contribution of this project is supposed to be the design and implementation of a service application that provides a modular, extensible, and reproducible framework for defining, executing, and analyzing experimental pipelines for recommender systems with sparse autoencoders. The application aims to automate common experimental workflows and to offer a web-based interface for managing experiments and inspecting results. By doing so, it reduces the overhead associated with research prototyping and evaluation and enables more systematic investigation of alternative approaches within the LLM4MORS research agenda.

¹The information about why the item was recommended by the complex model

²Guiding or influencing recommendation outcomes

1.1 Relation to other projects

This project directly builds upon prior work conducted within the LLM4MORS research initiative, particularly research focused on integrating sparse autoencoders into collaborative filtering models to improve interpretability and enable steering of recommendations. A key inspiration is the manuscript *“From Knots to Knobs: Disentangling Collaborative Filtering Autoencoders with Sparse Autoencoders”* (submitted to WWW 2026), in which the concept of augmenting recommender systems with sparse autoencoders was introduced for the first time. The manuscript proposes a novel architectural approach in which a sparse autoencoder is integrated into a collaborative filtering autoencoder in order to expose interpretable latent representations. This architecture is illustrated in Figure 1, which contrasts a standard collaborative filtering autoencoder with its extension incorporating an SAE hook.

Another closely related effort is the SAGEA project, which resulted in the research paper *“SAGEA: Sparse Autoencoder-based Group Embeddings Aggregation for Fairness-Preserving Group Recommendations”*. This work applies sparse autoencoders in the context of group recommender systems, demonstrating their usefulness for constructing interpretable and fairness-aware group representations. While the application domain differs from individual recommendation, the project further motivates the use of SAE as a general tool for representation learning and interpretability in recommender systems.

Existing projects within LLM4MORS and related work typically implement experimental pipelines in a tightly coupled and project-specific manner, often tailored to a single architectural choice or evaluation setup. As a consequence, extending these solutions to alternative configurations or systematically comparing different approaches requires substantial reimplementation effort. The present project aims to abstract and generalize these prior efforts by providing a reusable experimental framework that supports multiple architectures, training strategies, and evaluation methods, thereby enabling more efficient cooperation and methodological reuse among team members.

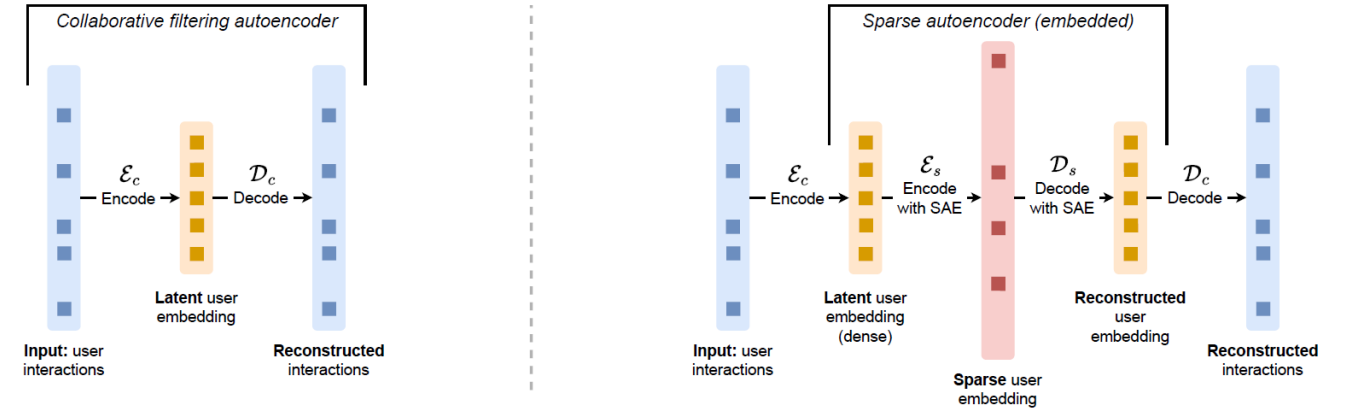


Figure 1: Left: Generic collaborative filtering autoencoder (CFAE) with a bottleneck layer. Right: CFAE with inserted SAE hook.

2 Project description

The objective of this research project is to develop an application that implements a framework for creation, execution, and evaluation of multi-step experimental pipelines for recommender systems employing sparse autoencoders. Each pipeline consists of sequence of configurable steps, such as model training, neuron labeling, evaluation, and steering, implemented via a plugin-based architecture. Upon execution, each step of the multi-step pipeline, produces intermediate results that are presented to the user, enabling inspection and analysis of individual stages of the pipeline through appropriate statistics or visualizations.

As part of the research contribution, at least one baseline plugin will be implemented for each step of the multi-step pipeline. These baseline implementations will provide reference solutions for training, labeling, evaluation, and steering, and will serve both as working defaults and as points of comparison for future experimental extensions.

The specific steps of the pipeline are following:

Data Loading. This step is responsible for loading the input dataset and preparing it for subsequent processing. It includes dataset selection or upload, basic preprocessing, and the definition of the training and evaluation setup, such as data splits and configuration of input features. In addition, this step may incorporate dataset augmentation or modification procedures aimed at achieving specific data distributions or properties, which can be beneficial for controlled experimentation and analysis in subsequent pipeline steps.

Training Collaborative Filtering Autoencoder. In this step, a collaborative filtering autoencoder is trained on the prepared dataset to learn latent representations of users based on observed interaction data. The trained model serves as the base recommender system and produces standard recommendation evaluation metrics.

Training Embedded SAE. This step trains a sparse autoencoder embedded within the collaborative filtering model. The SAE is used to obtain sparse and more interpretable, granular (disentangled) latent representations, and its performance is evaluated using appropriate efficiency and reconstruction-based metrics.

Neuron Labeling. The neuron labeling step aims to interpret individual neurons of the sparse autoencoder by associating them with human-understandable concepts. This is achieved by analyzing neuron activations with respect to data characteristics or metadata, resulting in a neuron dictionary or mapping.

Labeling Evaluation. In this step, the quality and consistency of the obtained neuron labels are evaluated at a global level, focusing on the overall structure of the produced label dictionary or its aggregated representations (e.g., clusters of labeled neurons). This evaluation provides insight into the coherence and usefulness of the labeling process as a whole.

Inspect & Steer. The final step operates at a finer level of granularity and enables detailed analysis of individual neurons and their associated labels. The *Inspect* phase allows users to examine a specific neuron by highlighting items or interactions for which the neuron is strongly activated. The *Steer* phase explores the effect of explicitly modifying the activation or contribution of selected neurons, enabling analysis of how such interventions influence recommendation behavior and outcomes.

A central aspect of the project is the **formalization of a unified plugin API** that defines how individual pipeline steps are implemented and executed. This API will specify how plugins receive inputs, access shared context produced by preceding steps, and produce outputs that can be consumed by subsequent steps in the pipeline. Through this mechanism, the entire pipeline, composed of multiple plugins, can be executed as a single experiment, while preserving explicit data and model dependencies between steps—for example, enabling a neuron labeling plugin to access a trained model or allowing an evaluation plugin to consume labeling results.

The plugin-based architecture is designed to support extensibility and experimentation. Researchers using the system will be able to implement custom plugins for individual pipeline steps and integrate them into experimental pipelines without modifying the core application. This design enables rapid prototyping of new methods and facilitates systematic comparison between baseline and user-defined approaches within a unified experimental framework.

While core functionalities will be accessible via a command-line utility, the primary **user interface will be provided as a web application**. In addition to enabling users to create, manage, and execute pipelines composed of plugins, the application will support loading previously executed pipeline runs and modifying or extending their individual components in order to compare the effects of changes in pipeline configuration. The system will provide structured storage of executed pipelines together with their intermediate and final results, offering an efficient and intuitive interface that allows researchers to easily navigate past experiments and systematically analyze and compare their outcomes.

The goal of the project is to prepare an experimental environment for the development and validation of pipelines. This will enable further exploration in this research domain by the research group and particularly by the solver, who plans to base his master’s thesis on this project, leveraging the framework as a practical tool.

2.1 Functional requirements

The system shall provide the following functional requirements:

- **F1: Plugin-based Pipeline Execution** The system shall support dynamic loading and execution of pipeline steps implemented as plugins, enabling flexible composition of experimental pipelines without changes to the core application.
- **F2: Pipeline Definition and Management** The system shall provide an interface for creating, configuring, and managing experimental pipelines composed of multiple ordered steps, including selection and parameterization of individual plugins.
- **F3: Experiment Execution and Tracking** The system shall enable execution of experimental pipelines on predefined datasets and shall store experiment configurations, execution metadata, and results for subsequent inspection.
- **F4: Results Inspection and Visualization** The system shall provide an interface for inspecting results and logs of executed pipeline runs, including visualization of selected outputs such as evaluation metrics and neuron interpretations.

- **F5: Filtering and Navigation Support** The system shall support efficient navigation and filtering of previously executed pipeline experiments, allowing users to search, group, and compare runs based on configuration parameters, execution metadata, or results.
- **F6: Reproducibility Support** The system shall support reproducibility of experiments by preserving pipeline definitions, plugin configurations, parameter values, and versioned artifacts associated with each execution.

Research-oriented Functional Requirements. Baseline plugin implementations will be provided for all major steps of the multi-step pipeline. For earlier pipeline steps, such as model training or neuron labeling, these baseline implementations are relatively straightforward, as suitable methods and practices are well established in prior research. In contrast, the following requirements address pipeline steps that are less explored and therefore require a more exploratory and design-oriented approach.

- **F6: Baseline Labeling Evaluation** The system shall include a baseline plugin for the labeling evaluation step that enables quantitative assessment of the quality of the produced neuron labeling dictionary or its aggregated representations.
- **F7: Baseline Inspect and Steering Support** The system shall include a baseline plugin for the inspect and steer step that provides informative outputs allowing users to assess the impact of neuron-level inspection and steering on the overall behavior and quality of the recommender pipeline.

2.2 Non-Functional requirements

The system shall satisfy the following non-functional requirements:

- **N1: Modularity and Extensibility** The system shall be designed in a modular and extensible manner, allowing new pipeline steps and plugin implementations to be added, replaced, or removed without requiring changes to the core application.
- **N2: Sufficient Performance** The system shall provide sufficient performance for research-scale experiments, enabling execution of computationally intensive pipeline steps within reasonable time limits.
- **N3: Deployability** The system shall be deployable in a Linux-based environment, including university or cloud infrastructure, and shall support containerized deployment. At the same time, the system shall remain runnable in a local environment on a standard personal computer.
- **N4: Multi-user Support** When deployed, the system shall support concurrent usage by multiple users and provide mechanisms for managing access to experiments and stored results.
- **N5: Documentation and Maintainability** The system shall be accompanied by comprehensive documentation describing its architecture, data flow, configuration options, and extension points. The codebase shall be maintainable and structured to support long-term use and further research development.

2.3 Mockups

The application is expected to provide a web-based user interface for creating, configuring, and executing experimental pipelines (see Figure 2). Through this interface, users will be able to construct pipelines composed of multiple sequential steps, where each step can be configured independently. For each pipeline step, the user can select one of the available plugin implementations and specify its corresponding parameters, allowing full control over the experimental configuration.

The mockup displays a web application interface for creating pipeline experiments, organized into three main panels: Training CFM, Training SAE, and Neuron Labeling. Each panel includes a 'Set up new' button (highlighted in blue) and a 'Load from previous run' button. The Training CFM panel features a 'Configure' button for the 'ELSA Trainer' (selected) and input fields for Epochs (64), Batch Size (100), and Factors (256). The Training SAE panel has 'Configure' buttons for 'Basic SAE', 'TopK SAE' (selected), and 'BatchTopK SAE'. The Neuron Labeling panel has 'Configure' buttons for 'TF-IDF' (selected) and 'LLM'. A 'Run up to this step' button is located at the bottom of each panel. A large blue 'Run full pipeline' button is positioned at the bottom of the entire interface.

Figure 2: Web application UI for creating pipeline experiments

In addition to creating pipelines from scratch, the interface will support reusing selected steps and their outputs from previously executed pipeline runs by loading them from persistent storage. This functionality is intended to streamline the experimentation process by enabling users to build upon existing results, avoid redundant computations, and focus on exploring the impact of targeted modifications within the pipeline.

The application will further provide an interface for browsing and inspecting results of completed pipeline runs (see Figure 3). Executed experiments will be presented in a clear and intuitive manner, enabling users to easily navigate between individual runs, inspect their configurations, and compare outcomes across different pipeline variants. This overview is designed to support systematic analysis and comparison of experimental results across multiple executions.

Run Name	Created	Dataset	Duration	User	Source
pipeline_run_2025-12-14_17-20-44	13 days ago	-	4.0min	jan-kratky	__ma
neuron_labeling_tf_idf	13 days ago	-	58.4s	jan-kratky	__ma
training_sae_sae_trainer	13 days ago	-	52.6s	jan-kratky	__ma
training_cfm_elsa_trainer	13 days ago	-	2.1min	jan-kratky	__ma
pipeline_run_2025-12-14_17-08-52	13 days ago	-	5.2min	jan-kratky	__ma
neuron_labeling_tf_idf	13 days ago	-	1.3min	jan-kratky	__ma
training_1_sae_trainer	13 days ago	-	1.1min	jan-kratky	__ma
training_1_elsa_trainer	13 days ago	-	2.8min	jan-kratky	__ma
pipeline_run_2025-12-13_20-56-01	14 days ago	-	5.4min	jan-kratky	__ma
pipeline_run_2025-12-13_20-33-19	14 days ago	-	3.8min	jan-kratky	__ma

Figure 3: UI for displaying previous pipeline experiments (for this purpose the configured UI for MLflow server could be utilized)

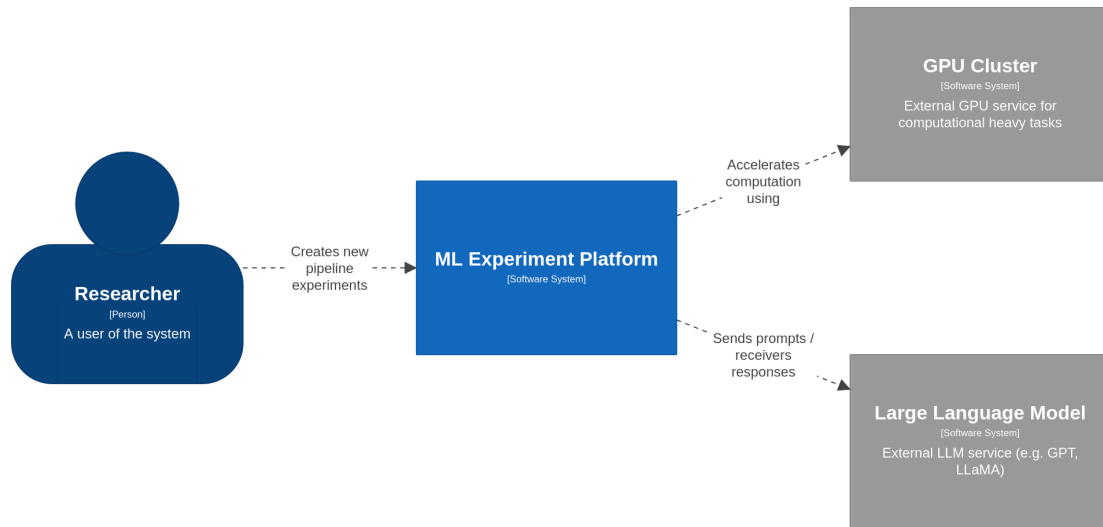
2.4 Architecture

The architecture of the proposed system is described using the C4 model, which provides a structured, multi-level view of the system and its components. Figures 4, 5, and 6 illustrate the System Context, Container, and Component views, respectively.

The *System Context view* (Figure 4) presents the system as a whole and its interaction with external actors and services. The primary user of the system is a researcher who creates and executes experimental pipelines. The system interacts with external computational resources, such as a GPU cluster for accelerating computationally intensive tasks, and with external large language model services used in selected pipeline steps.

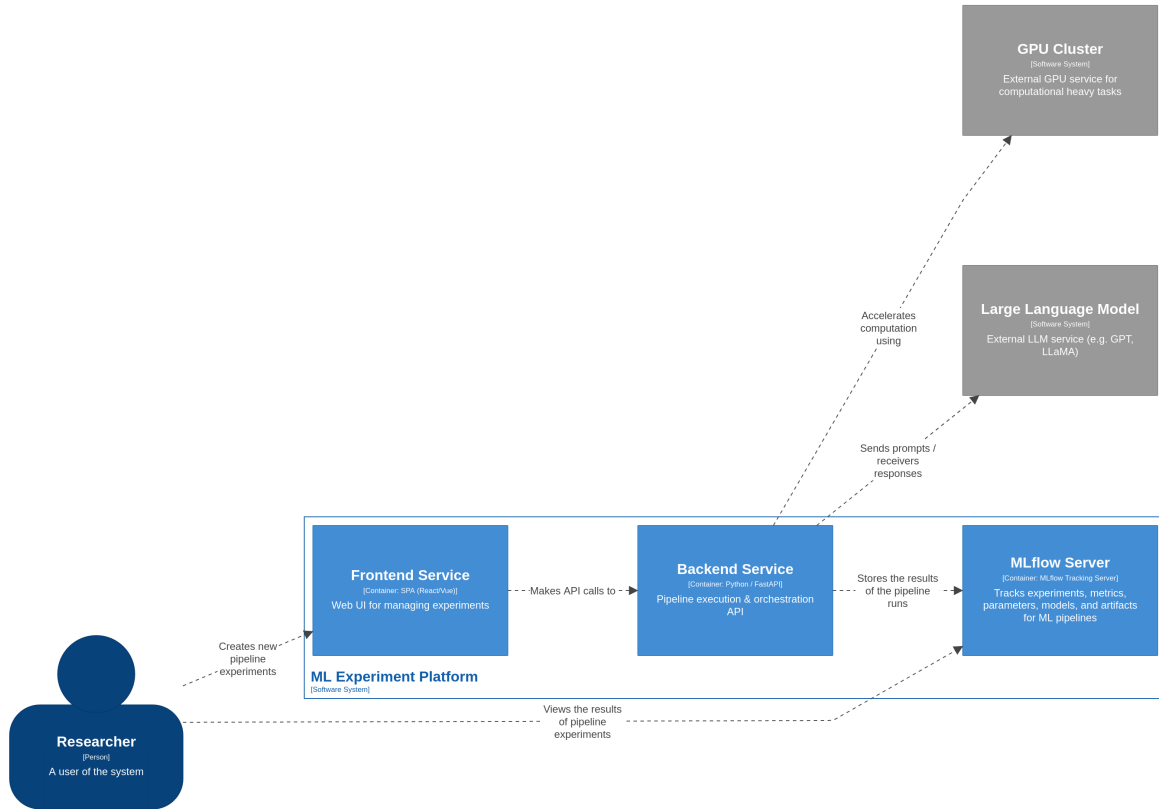
The *Container view* (Figure 5) decomposes the system into its main runtime containers. The system consists of a frontend service providing a web-based user interface for managing experiments and a backend service responsible for pipeline orchestration and execution. The backend communicates with an MLflow tracking server, which is used for storing experiment metadata, parameters, metrics, and artifacts. This view emphasizes the separation of concerns between user interaction, execution logic, and experiment tracking.

The *Component view* (Figure 6) further details the internal structure of the backend service. The core of the backend is the pipeline engine and plugin manager, which are responsible for loading, executing, and coordinating individual pipeline steps implemented as plugins. Supporting components handle experiment tracking through MLflow, interaction with external GPU infrastructure, and communication with external large language model services when required. This view highlights the plugin-based architecture of the system and illustrates how individual pipeline steps are integrated into a unified execution framework.



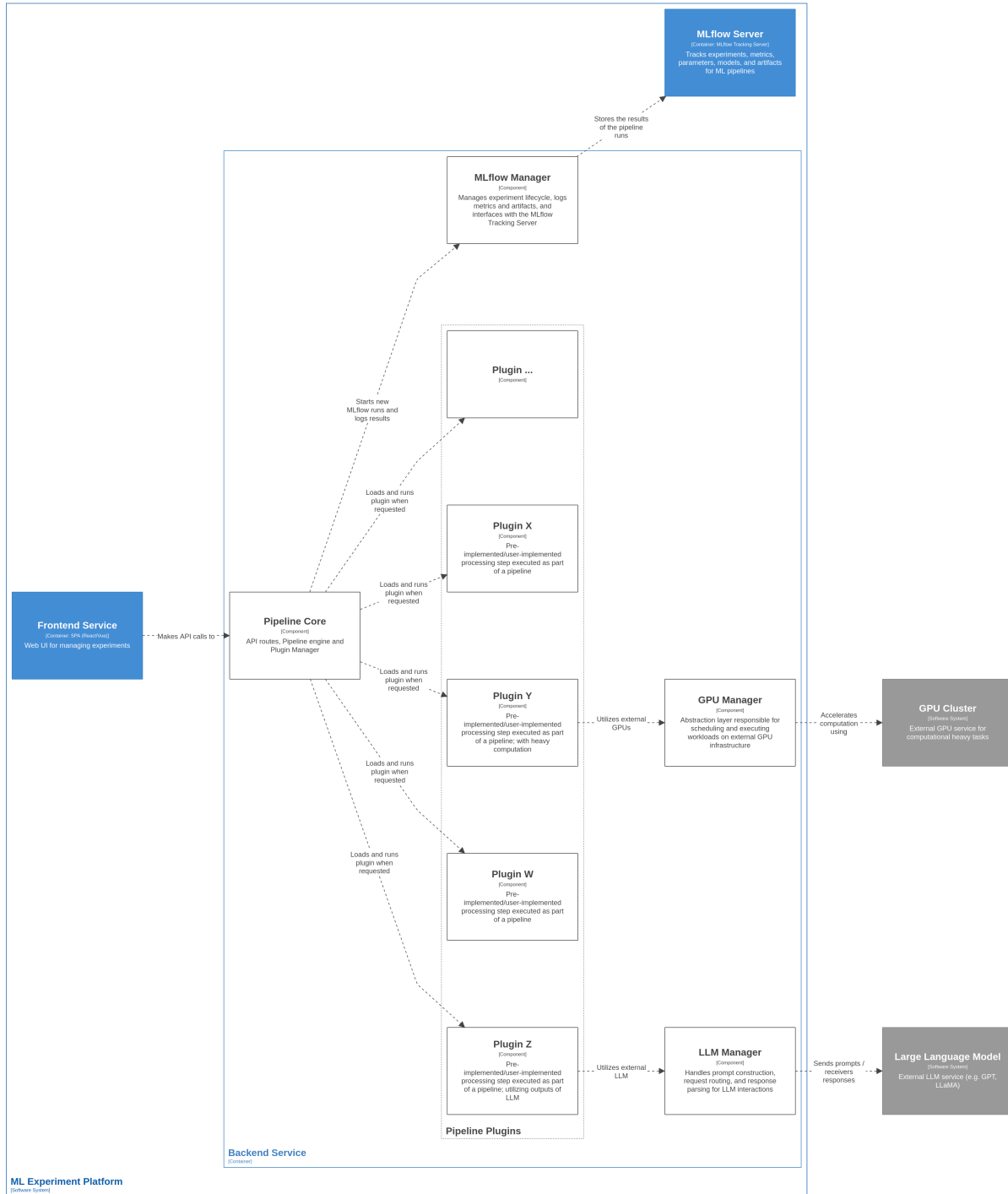
System Context View: ML Experiment Platform
Monday, December 29, 2025 at 12:41 PM Central European Standard Time

Figure 4: Project Architecture - System Context View



Container View: ML Experiment Platform
Monday, December 29, 2025 at 12:41 PM Central European Standard Time

Figure 5: Project Architecture - Container View



Component View: ML Experiment Platform - Backend Service
Monday, December 29, 2025 at 12:41 PM Central European Standard Time

Figure 6: Project Architecture - Component View

3 Platform and technologies

Planned technologies include:

- Programming language: Python, JavaScript
- Backend framework: FastAPI
- Frontend: React (or equivalent modern framework)
- Experiment tracking: MLflow
- Data processing and ML: NumPy, PyTorch, Pandas, Polars
- Visualization: Plotly or D3.js
- Version control: Git
- Deployment: Docker

4 Evaluation

The software will be evaluated primarily through experimental validation. Evaluation criteria include:

- **Correctness and stability of pipeline execution.**
- **Reproducibility of experimental results.**
- **Ability to replicate selected results from existing research.**
- **Usability of the web interface for managing experiments.**

The evaluation of these criteria will be carried out by members of the research group, primarily the supervisor, reflecting the fact that the developed application is intended as a research tool supporting their ongoing work. The results of this evaluation will be presented as part of the project outcomes.

5 Risks

The project involves several risks related to its research-oriented nature and the evolving state of the application domain. The main risks and their mitigation strategies are outlined below.

- **Changing research requirements.** As the field of interpretable recommender systems and sparse autoencoders is actively researched within the LLM4MORS group, research requirements may evolve during the project. For example, the set of baseline plugins for individual pipeline steps may change. This risk is mitigated by the plugin-based architecture, which supports extensibility and allows new methods to be integrated without modifying the core system.
- **Integration complexity.** Pipeline steps may differ in their data, model, and evaluation requirements, increasing integration complexity. This risk will be addressed through a clearly defined plugin API and a shared execution context governing data and model exchange between steps.

- **Time constraints.** Experimentation and evaluation may be time-consuming, particularly for computationally intensive models. The project therefore follows an incremental development approach with early prototyping and baseline implementations, allowing prioritization of essential functionality if needed.

6 Milestones / Deliverables

The main deliverable of the project is a **functional service application** that provides a modular framework for creating, executing, and analyzing experimental pipelines for recommender systems employing sparse autoencoders. The delivered system will support end-to-end execution of multi-step pipelines and will demonstrate the feasibility of the proposed architecture.

The application will be deployed within the university infrastructure. At the same time, the system will be designed to be easily runnable in a local environment on a standard personal computer, allowing researchers to use and extend the application independently of the deployment environment.

The project deliverables will further include the **complete source code** of the application maintained in a version-controlled repository, together with **comprehensive documentation**. The documentation will describe the system architecture, deployment and execution instructions, and guidelines for defining experimental pipelines and implementing custom plugins.

The project will be presented to the commission in the form of a live or recorded demonstration showcasing the creation, execution, and analysis of experimental pipelines using the developed application. Access to the source code repository will be provided to the commission; if the project is not publicly accessible, read-only access will be granted upon request.

7 Time Schedule

The following time schedule presents an approximate plan of the project activities and milestones, which may be refined during the course of the project.

Milestone / Deliverable	Activity	Time schedule	MDs
Specification and Architecture Design	Studying the use of SAEs in recommender systems, reviewing previous group projects, designing the overall solution architecture, and conducting preliminary research	Nov 25	12
Prototype	Implementation of backend core components, pipeline engine, and MLflow integration	Dec 25	8
	Baseline plugins implementation and formalization of the plugin API	Dec 25 – Jan 26	12
	Minimal frontend (web UI for pipeline creation and execution)	Jan 26 – Feb 26	6
End-to-End Application	Design and implementation of standalone, self-contained plugins for all pipeline steps	Feb 26 – Mar 26	14
	Integration with external and additional services (e.g., GPU infrastructure, LLM services)	Mar 26	6
	Implementation of a fully functional frontend for experiment management and result inspection	Mar 26 – Apr 26	10
	Functional and end-to-end testing of the complete application	Apr 26	6
Deployment	Deployment of the application within university infrastructure and preparation of local execution setup	Apr 26	4
Documentation	Preparation of user and developer documentation, including architecture description and usage guidelines	Apr 26 – May 26	5
Evaluation	Experimental evaluation of the application and subsequent refinement based on evaluation results	May 26	4

8 Form of collaboration

The collaboration within the project will be structured through a combination of regular meetings and ad-hoc consultations with the supervisor and project consultants.

8.1 Consulting plan

Regular group meetings within the GAČR LLM4MORS project are planned on a bi-weekly basis. During these meetings, individual team members report on their progress over the preceding two weeks and discuss ongoing research and development activities. These meetings provide an opportunity to consult the project with multiple consultants, receive feedback on intermediate results, and coordinate work across related research tasks.

In addition to group meetings, individual consultations with the supervisor are planned every two weeks, offset by one week relative to the group meetings. These one-on-one consultations focus specifically on the progress of

this project, including detailed discussion of design decisions, implementation challenges, experimental results, and planning of subsequent steps.

8.2 Collaboration with other team members

Collaboration with other members of the research group will be tailored to their respective areas of expertise. In particular, consultations with Mgr. Martin Spišák will focus on methodological aspects related to the training of collaborative filtering models and sparse autoencoders, as well as neuron labeling techniques. Petr Škoda Ph.D. and RNDr. Patrik Dokoupil will be consulted primarily on infrastructural and functional aspects of the project, including system design, deployment considerations, and robustness of the implemented solution.

The supervisor, Mgr. Ladislav Peška, Ph.D., will provide guidance on overall project progress, research direction, and alignment with the objectives of the LLM4MORS initiative. All consultants will also be involved on an ad-hoc basis, particularly in discussions related to usability of the developed application, evaluation of experimental workflows, and refinement of the framework based on emerging research needs.